

From Lux AI S3 to Game-Theoretic Layering

A First-Principles Problem Analysis of a Partially Observable Multi-Agent Competition (Part 1)

Bo Hou | jsjhoubo.github.io

This note only does the “reading of the problem”: formalizing the rules precisely and clarifying the layered game structure. Modeling and solving are deferred to later parts.

1 Why Start With Reading the Problem

Lux AI Season 3 is an official NeurIPS 2024 competition: two teams each control a squad of units on a 24×24 grid, scoring by occupying hidden “point tiles”, with a best-of-five format deciding the match. It is deliberately designed around partial observability and meta-learning: each episode’s rule parameters and point-tile locations are unknown at the start, and a strong policy must infer them *while playing*.

Treating it as an interview problem, the first step is not to write an agent but to **read the problem precisely**—translate every rule into mathematics, then see clearly *what kind of game* this actually is. This note records that step. All mechanics are verified line-by-line against the official source (`env.py` / `state.py` / `params.py` in `luxai_s3`).

2 A Mathematical Statement of the Game Facts

Time and space. An episode consists of $K = 5$ matches, each of $H = 100$ steps; global time $\tau = (k-1)H + t$. The map is $\mathcal{C} = \{0, \dots, 23\}^2$ with cells $c = (x, y)$. Two teams $p \in \{0, 1\}$, each with at most $N_{\max} = 16$ units.

Units and actions. A unit’s state is $u_{p,i}^{(\tau)} = (\text{alive}, \mathbf{pos}, e)$ with energy $e \in [0, 400]$, initialized at 100. Spawn corners are hard-coded at the two diagonal points $c_0 = (0, 0)$ and $c_1 = (23, 23)$; one new unit is produced every 3 steps, stacked on the corner. An action is $a = (m, \delta x, \delta y)$: $m \in \{0, 1, 2, 3, 4\}$ for movement (stay / up / right / down / left, one cell per step), and $m = 5$ for sap (a ranged energy strike).

Hidden parameters. $\theta = (\mathcal{R}, \{\mathcal{G}_r\}, \rho)$ is fixed across the 5 matches, unknown at the start, and is the target of meta-learning:

- $\mathcal{R}$: relic-node positions, $|\mathcal{R}| \leq 6$ (actually $1 \sim 3$ active plus a mirror-symmetric copy).
- $\mathcal{G}_r$: a hidden 5×5 point-tile mask M_r around each relic, each cell iid Bernoulli(≈ 0.2)—about 5 of the 25 cells score.
- ρ : a vector of physical parameters (move_cost, sensor_range, sap_cost/range, nebula vision and energy reduction, etc.), drawn each episode from finite sets—it defines “the physics of this particular episode”.

Scoring (de-duplicated per cell). Let $\text{occ}_p^{(\tau)}(c) = \mathbb{1}[\exists i : \text{an alive unit of } p \text{ is on } c]$ and $\text{relic}(c) = \mathbb{1}[c \in \bigcup_r \mathcal{G}_r]$. The instantaneous score is

$$g_p^{(\tau)} = \sum_{c \in \mathcal{C}} \text{relic}(c) \cdot \text{occ}_p^{(\tau)}(c).$$

A point tile scores +1 regardless of how many units stand on it; overlapping point tiles still count once. For a cell covered by m relics’ 5×5 regions, the prior is $P(\phi_c = 1) = 1 - 0.8^m$.

Win conditions. A match is won by the higher cumulative $\sum_t g_p$ (ties broken by total energy, then randomly); the episode is won by the greater number of matches won.

Combat resolution. Sap: full `sap_cost` damage on the center cell, discounted by `dropoff` on the 8 neighbors, with multi-unit focus fire adding linearly (a unit saps at most once per step; “multiple hits” come from multiple units focusing one target). Collision: units of opposing teams on the same cell compare their *sum of energy on that cell*; the lower side is entirely removed, ties remove both—and *this is resolved before scoring*, so the winner clears the opponent off the cell and then scores, but the kill itself grants no points. Void field: applies to the four orthogonal neighbors, does not decay with distance, scales with the enemy unit’s energy, and is divided by one’s own stack size on the cell.

Key information fact. In the observation, `team_points` has shape 2 and is *not masked*—**both teams’ totals are mutually public**; unit positions/energy are partially visible under the sensor mask, and the point-tile truth is fully hidden. The opponent’s public score changes are therefore an additional channel for inferring θ .

3 The Information Structure: the Single Axis of Layering

Layering uses one axis only—**what both players know about the point tiles ϕ** . That the opponent’s total is public, units are partially visible, and the opponent’s information state is hidden, are environment-given constants, not layering axes. The referee (the system) is omniscient, does not compete, and does not enter the layering.

Layer	Both players’ knowledge of ϕ	Game type
L1	Known	complete-information, simultaneous-move two-player Markov game
L2	Unknown (inferred from totals)	incomplete-information two-player game (the real Lux)

L0 is not an independent layer. “Inferring ϕ from the totals” is a *tool* used inside L2 by each player: the cells ϕ_c are independent a priori, but an observation $y = \sum_{c \in S} \phi_c$ is a sum over a set of cells, so the likelihood does not factorize and the posterior *couples* the jointly-observed cells—the structure of group testing / compressed sensing. L0 serves L2 and is not in the same category as L1/L2; treating it as a sibling is what creates the illusory “blurry boundary between L0 and L1”.

The globally optimal policy targets L2, but the solution path is $L0 \rightarrow L1 \rightarrow L2$, assembled layer by layer. A rule-based agent does explicit mathematical assembly; RL approximates the same L2 optimum end-to-end—the two align because they face the *same information bottleneck* (only the totals are visible).

4 A Modeling Choice That Decides the Difficulty: Centralization

A team has many units—does it count as “one brain controlling many units” or “many independent agents”? This is not forced by the game but is an implementation choice, and it directly decides the complexity class:

Modeling	Decision-makers per team	Overall model
Centralized (one brain)	1 (large joint action space)	two-player POSG / POMDP vs. fixed opponent
Decentralized (independent units)	N	Dec-POSG

Lux’s `act` receives the *whole team’s* observation and outputs the *whole team’s* action—so centralization is allowed. Adopting it reduces the problem from a decentralized Dec-POMDP (NEXP-complete) to a two-player POSG ($\sim$ PSPACE), and keeps $L1 \rightarrow L2$ a *two-player* game throughout, with a continuous framework. The cost is a joint action space $\leq 6^{16}$ —a *computational* problem (addressable by decomposition / heuristics),

not the *structural* hardness of Dec-POMDP. In other words, centralization trades “multi-agent coordination is hard” for “the action space is large”, and the latter is far more tractable.

Complexity placement. The exact optimum lies in PSPACE / NEXP and is unattainable. The goal is therefore not a global Nash equilibrium but *structural insight* + *solvable sub-games* + *high-quality approximation*—the same pragmatism by which AlphaGo does not “solve” Go (EXPTIME) but approximates it with learning and search.

5 The Game-Theoretic Framework for L1

A game $\Gamma = \langle \mathcal{N}, \{A_i\}, \{u_i\} \rangle$. Nash equilibrium: an outcome (a_1^*, a_2^*) at which neither player can improve *their own* utility by deviating unilaterally,

$$u_1(a_1^*, a_2^*) \geq u_1(a_1, a_2^*) \forall a_1, \quad u_2(a_1^*, a_2^*) \geq u_2(a_1^*, a_2) \forall a_2.$$

Both conditions point the *same direction* for both players (each maximizes its own u_i)—keeping this form prevents flipping an inequality in the zero-sum “single-function max / min” translation.

Checking vs. inferring. *Checking* whether an outcome is an equilibrium needs only the definition, verified condition by condition, and is never misused. The real “misuse” happens in *inference*: applying properties like “the value is unique, equals the minimax, predicts the winner, is optimal” directly, without verifying their precondition (zero-sum or not). The discipline: whenever you use an equilibrium property, return to its precondition and prove it, rather than invoking a remembered conclusion.

The true structure of L1: non-zero-sum scoring + zero-sum verdict. The scoring process is *non-zero-sum*—both sides occupy mutually non-conflicting cells, $G_A + G_B$ is not constant, and A gaining a cell does not necessarily cost B one; only contesting the *same* cell is *locally* zero-sum. But the final verdict $\mathbb{K}[G_A > G_B]$ is zero-sum. Each side’s true objective is thus a composite: maximize one’s own occupation efficiency (non-zero-sum) and suppress the opponent in contests (zero-sum). Since the verdict depends only on the sign of the score difference, *excess points are worthless*—“defend when ahead, gamble when behind” follows naturally.

6 A Derived Sub-Result: Single-Cell Contest (one-step 2×2)

The minimal combat unit: one point tile g , each side with one unit already adjacent to g , simultaneously deciding to step in (I) or not (N).

Utility contains only instantaneous score. In the source, each step’s reward comes only from occupation; kills, energy changes, and unit losses *do not enter the immediate reward*—they belong to the state layer / future value V . Although collision resolves before scoring, it merely clears the opponent off the cell so the winner can occupy it; the kill itself adds nothing. Let u be player 1’s score difference ($u_1 - u_2$, zero-sum on a single cell):

Energy	$u(I, I)$	$u(I, N)$	$u(N, I)$	$u(N, N)$	pure NE (value)
P1 higher	+1	+1	−1	0	$(I, I), (I, N)$ (+1)
Equal	0	+1	−1	0	(I, I) (0)
P1 lower	−1	+1	−1	0	$(I, I), (N, I)$ (−1)

When P1’s energy is higher, I is strictly dominant; when equal, (I, I) is the unique equilibrium (not stepping in lets the opponent take the cell for free, so one steps in even knowing it means mutual destruction—a deterrence equilibrium); when lower, contesting loses either way. The two equilibria share the same value—exactly as “all

NE of a zero-sum game share one value”, which conversely confirms that the single step is indeed zero-sum and that no unit value w should be mixed into the utility.

A usable strategic principle: contest a cell only if $e_{\text{self}} \geq e_{\text{opp}}$; when at an energy disadvantage, do not contest—conserve the unit for uncontested cells. Unit value w and energy consumption belong to the future value of a *multi-step* Bellman recursion, not the single-step matrix—a separation worth remembering: *the immediate layer holds only occupation score (zero-sum), while the state layer (energy, survival) affects the future V .*

7 Next (Part 2 Preview)

- **One-step A+C:** multiple cells + focus fire, a *graph-constrained local allocation game*—units can only step into cells reachable in one move, so it is *not* the freely-allocable classic Colonel Blotto. Minimal nontrivial case: the binary choice of a unit that can reach two cells.
- **Blotto:** the classic version (why no pure-strategy equilibrium exists; discrete vs. continuous) and Lux’s graph-constrained variant.
- **Multi-step (B):** introducing travel, with a point tile’s value $\sim H - d$.
- **L0:** the information-theoretic lower bound of group testing / compressed sensing— $H(\phi)$ against $\log_2(s+1)$ bits per observation.

All mechanics verified against the official source; the formalization and layering are the author’s own derivation. Corrections welcome.